LAB EXPERIMENTS

1.A class consists of 40 students who have secured different grades in an examination. The grades are classified as F, E, D, C, B, A, and S, where F represents the lowest grade and S represents the highest grade.

AIM

To develop a Python program in Google Colab to represent the grade-wise distribution of 40 students using a pie chart and a bar chart, identify the maximum and minimum student counts, and calculate the combined percentage of students who secured grades A and S.

ALGORITHM

1. Start the program.
2. Import the matplotlib.pyplot library.
3. Store the grades (F, E, D, C, B, A, S) and their corresponding student counts in lists.
4. Calculate the total number of students and the percentage of students in each grade.
5. Draw a pie chart to represent the percentage distribution of students, displaying grade labels and percentage values.
6. Draw a bar chart to compare the number of students in each grade.
7. Label the X-axis as Grades and the Y-axis as Number of Students.
8. Display the exact number of students above each bar and add suitable chart titles and legends.
9. Identify and display the grades with the maximum and minimum numbers of students.
10. Calculate and display the combined percentage of students who secured grades A and S.
11. Stop the program.

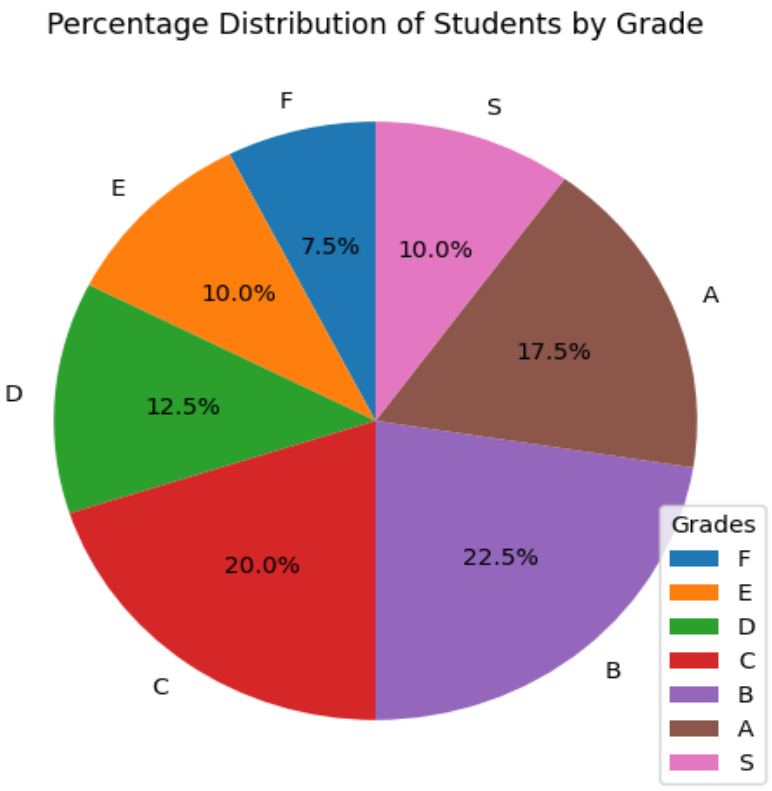
PROGRAM

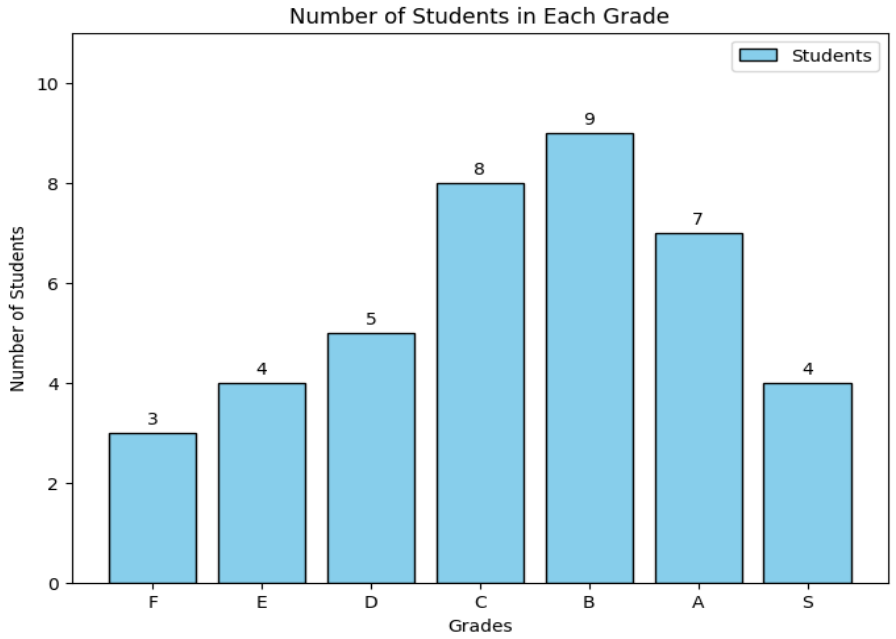
```
import matplotlib.pyplot as plt
# 1. Store grades and number of students
grades = ['F', 'E', 'D', 'C', 'B', 'A', 'S']
students = [3, 4, 5, 8, 9, 7, 4]
total_students = sum(students)
# Calculate percentages
percentages = [(n / total_students) * 100 for n in students]
# 2 & 3. Draw a pie chart with labels and percentages
plt.figure(figsize=(8, 6))
plt.pie(
    students,
```

```
    labels=grades,
    autopct='%1.1f%%',
    startangle=90
)
plt.title('Percentage Distribution of Students by Grade')
plt.legend(grades, title='Grades', loc='best')
plt.show()
# 4, 5 & 6. Draw a bar chart with values above each bar
plt.figure(figsize=(8, 6))
bars = plt.bar(grades, students, color='skyblue', edgecolor='black')
plt.xlabel('Grades')
plt.ylabel('Number of Students')
plt.title('Number of Students in Each Grade')
# Display exact number above each bar
for bar in bars:
    height = bar.get_height()
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height + 0.1,
        str(int(height)),
        ha='center',
        va='bottom'
    )
plt.legend(['Students'])
plt.ylim(0, max(students) + 2)
plt.show()
# 8. Identify maximum and minimum grades
max_students = max(students)
min_students = min(students)
max_grades = [
    grades[i] for i in range(len(grades))
    if students[i] == max_students
]
min_grades = [
    grades[i] for i in range(len(grades))
    if students[i] == min_students
```

```
]
print("Total number of students:", total_students)
print("Grade(s) with maximum students:", max_grades,
      "-", max_students, "students")
print("Grade(s) with minimum students:", min_grades,
      "-", min_students, "students")
# 9. Percentage of students with A and S grades combined
a_s_students = students[grades.index('A')] + students[grades.index('S')]
a_s_percentage = (a_s_students / total_students) * 100
print("Students who secured A and S grades:", a_s_students)
print("Percentage of A and S grades combined:",
   round(a_s_percentage, 2), "%")
```

OUTPUT





Total number of students: 40

Grade(s) with maximum students: ['B'] - 9 students

Grade(s) with minimum students: ['F'] - 3 students

Students who secured A and S grades: 11

Percentage of A and S grades combined: 27.5 %

RESULT

The Python program was successfully executed in Google Colab to display the pie chart and bar chart representing the grade-wise distribution of students. Thus, the grade distribution was visualized successfully, and the required statistical results were obtained.

2.Semantic Network Using NetworkX

Aim

To represent concepts and labeled relationships as a directed semantic network.

Software Required: Python 3.x and NetworkX (install with: pip install networkx)

Algorithm

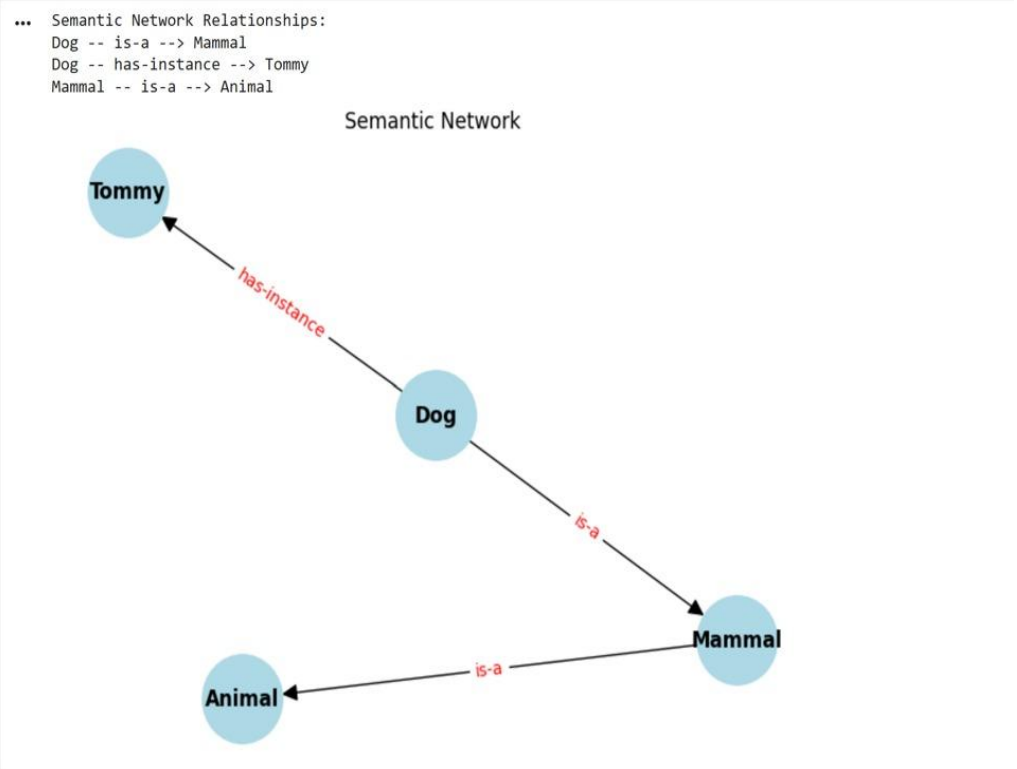
1. Start.
2. Import the networkx library.
3. Create a directed graph.
4. Add concepts as nodes through edges.
5. Add relationships between concepts, such as is-a and has-instance.
6. Display each relationship between the concepts.
7. Draw the semantic network graph.
8. Display the graph with labeled nodes and edges.
9. Stop

Program

```
!pip install networkx matplotlib
import networkx as nx
import matplotlib.pyplot as plt
# Create a directed graph
G = nx.DiGraph()
# Add concepts and relationships
G.add_edge("Dog", "Mammal", relation="is-
a")
G.add_edge("Mammal", "Animal",
relation="is-a")
G.add_edge("Dog", "Tommy", relation="has-
instance")
# Display relationships
print("Semantic Network Relationships:")
for source, target, data in G.edges(data=True):
 print(source, "--", data["relation"], "-->",
target)
# Draw the semantic network
```

```
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, with_labels=True,
node_color="lightblue",
 node_size=2500, font_size=10, arrows=True)
edge_labels = nx.get_edge_attributes(G,
"relation")
nx.draw_networkx_edge_labels(G, pos,
edge_labels=edge_labels)
plt.title("Semantic Network")
```

Expected Output



Result: The semantic network represented concepts and their relationship has been executed successfully.

3.Frame-Based Knowledge Representation

Aim

To represent an object as a frame containing slots and values.

Software Required: Python 3.x

Software Required: Python 3.x

Algorithm

1. Start.
2. Create a dictionary to represent a vehicle frame.
3. Store the vehicle details as slots and values.
4. Include slots such as class, brand, fuel type, wheels, and color.
5. Display the title "Vehicle Frame".
6. Use a loop to access each slot and its corresponding value.
7. Print all the slots and values.
8. Stop.

program

```
# Create a vehicle frame using a dictionary
vehicle = {
 "class": "Vehicle",
 "brand": "Toyota",
 "fuel_type": "Petrol",
 "wheels": 4,
 "color": "White"
}
# Display the frame
print("Vehicle Frame")
for slot, value in vehicle.items():
 print(slot, ":", value)
```

Expected output

```
Vehicle Frame
class : Vehicle
brand : Toyota
fuel_type : Petrol
wheels : 4
color : White
```

Frame-Based Knowledge Representation: Vehicle

Slot	Value
class	Vehicle
brand	Toyota
fuel_type	Petrol
wheels	4
color	White

Result: The Frame-Based Knowledge Representation has been executed successf